

黃子誠 (**ZI-CHENG HUANG** / **alias LanDX**)

聯絡電話 : +886 908-214-592

電子信箱 : zicheng.huang.dx@gmail.com

技術作品 : <https://github.com/johnny60904>

線上作品集 : <https://github.io>

求職戰略定位 / **EXECUTIVE SUMMARY**

具備 12 年以上連鎖零售第一線核心營運、危機處理與流程標準化經驗的高自律、高度自主驅動技術職人。擅長將「高密度資訊輸入 (High-Information-Density Input)」、「動態脈絡控制 (Dynamic Context Control)」、「對話控制訊號 (Conversation Control Signals)」、「對話工程 (Dialogue Engineering / Context Engineering / In-Context Alignment / Interactive Alignment / Conversational Prompting) (主動建模 Active Modeling)」之前沿生成式 AI 協同工程 (Human-AI Collaboration)，務實融入標準辦公室庶務、跨部門溝通與企業後勤優化工作流 (Workflow) 中。

擁有扎實的底層邏輯解構能力，在無 Dependency Injection (DI) 框架 (Framework) 的限制下，純手工以 Java 21、C# 與 PowerShell Core 實作出具備高防禦性、零記憶體配置 (Zero-Allocation)、以及高階架構可觀測性 (Advanced & Forensic Observability) 的系統組件與自動化流程腳本。

期望尋找一份作息健康固定、見紅就休的「連鎖品牌總部數位營運優化專員」或「企業高階行政流程自動化特助」崗位，以高度自律的專業素養與高效率科技產出，為團隊創造核心落地的數位轉型商業價值。

核心技術與專業競爭力 / **CORE COMPETENCIES**

一、高階人機協同與認知工程 (**Advanced Human-AI Co-Piloting**)

- 精通結構化提示詞架構 (**Prompt Architecture**)、對話控制訊號、抽象層級控制、與「會話狀態轉移 (**Session State Transfer**) 」技術，具備精準的脈絡控制力。
- 建立嚴謹的多輪批判性求證機制，能以高度理性的資訊密度主動捕捉並校正大型語言模型 (**LLM**) 的幻覺 (**Hallucination**)，將 **AI** 轉化為高效的知識萃取與工作流推進器。

二、跨平台流程自動化與防禦性腳本開發 (**Workflow Automation Engine**)

- 熟練運用 **PowerShell Core** 進行 **Class-Based** 的跨平台核心 **Cmdlet** 開發，具備對齊官方原生 **Cmdlet** 語意與行為設計 (如 **-Recurse** 遞迴/遍歷 **Parameter** 設計、支援 **Pipeline input/output**、等等) 的嚴格工程紀律。
- 具備因地制宜的路徑格式防禦思維，能有效辨識、防範、並清洗 **Windows**、**Linux**、**macOS** 三大作業系統非法 **File System Path** 的結構防呆，擅長編寫高安全規格、防範 **OOM** 併發/飽和攻擊之自動化批次處理腳本。

三、多層次雙語意架構診斷報錯系統 (**Forensic Exception Taxonomy**)

- 堅持嚴格的權責分離 (**SoC**) 與架構內聚性，為阻絕語意汙染，手工在 **Domain Layer** 與 **Application Layer** 分別實作出高度對稱、但語意各自精準的獨立例外系統 (**12 pillar Domain Invariant** 例外 vs **6 Pillar Inbound Request Contract** 拒絕例外)。
- 配合自訂的 **Bitmask** 錯誤碼生成器 (**Bitmask Error Code Generator**)，能輸出內含系統架構特徵 (**Workspace**、**Scope**、**Architectural Paradigms**、**Architectural Style**、**Architectural Pattern**、**Architectural Stereotype**、**Language Element**、...等等) 之高可觀測性錯誤訊息，大幅降低生產環境的維運除錯時間 (**MTTR**)。

四、底層代碼解構與高效能演算法優化 (**Low-Level Mechanics & Optimization**)

- 習慣優先理解事物底層運作機制。熟練掌握 **Git/GitHub** 版本控制 (含 **GPG/SSH** 數位安全簽章驗證)、**Nginx** 基礎網站部署與反向代理、**HTTP/RESTful** 協議基礎概念、**OAuth** 基礎概念、以及 **x86 Assembly** 及執行期記憶體數據分析。
- 在演算法實作中，具備微秒級效能與垃圾回收 (**GC**) 控管意識，手工寫出零記憶體配置 (**Zero-Allocation**) 的數字與字元拓撲防衛庫、以及基於 **Flyweight** 模式的延遲加載虛擬序列集合工廠。

工作經歷 / WORK EXPERIENCE

全家便利商店 (FamilyMart) | 門市核心營運與高併發客戶服務專家

2010 年 3 月 ~ 2026 年 3 月 (共 12 年+)

- 營運高併發與危機處理：長期深耕第一線零售高壓力環境，深諳高併發 (High-Concurrency) 下的客戶流量分流、客訴危機處理 (Crisis Management) 與供應鏈標準化營運流程。
- 高度自律與作息穩定：長期擔任固定早班核心職務 (07:00 - 15:00)，始終維持高品質、高自律且極度健康的日班生活作息。這段長達十年的穩定紀錄，是在長週期崗位上維持零出錯、高品質產出的營運護城河。
- 效率優化與流程重構：習慣在繁瑣的門市日常 (補貨、進銷存、盤點、報廢、帳務) 中，主動解構背後的底層流程機制，並透過資料整理與建立標準方法論，極大化日常庶務的執行效率，進而協助店鋪降低門市損耗。

數位轉型與軟體工程實戰成就 / SELECTED PROJECTS

項目一：

分散式遺留平台防腐層 (ACL) 建置專案 | 獨立開發者

專案網址：<https://github.com/johnny60904/legacy-platform-acl>

- 專案背景：針對一個具備 10 年以上歷史、缺乏設計模式、且含有複雜技術債 (包含 7500 行大型上帝類別 God Class、循環依賴 Cyclic Dependency、Persistence Corruption、時間溢位、等等) 的大型遺留核心系統 (Legacy Core Monolith)，手工在其旁邊建立現代化的集成防腐層 Anti-Corruption Layer。
- 架構設計：採用「Hybrid Architecture 混合架構 (Vertical Slice + DDD + Clean Architecture)」與純 Java 21。在沒有 Spring 等 DI 框架的 Frameworkless 架構限制下，利用無狀態的 Bill Pugh Singleton 單例模式手工實作出執行緒安全 (Thread Safe) 的組合根 (Composition Root) 與構造函數注入 (Constructor DI)。
- 防腐集成：設計「上下文綁定流式調和引擎 Immutable Fluent Mutators (`ExpirationReconciler`)」，利用 Java 21 record 載體與現代 Pattern-Matching Switch Expression 語法，在基礎設施邊界 (Infrastructure Boundary) 對老舊系統 (Legacy System) 不穩定的時間戳 (Timestamp) 進行數據清洗 (Sanitize) 與調和 (Reconcile)，將潛在異常在 Compile-Time 階段直接攔截。
- 法律合規：為了確保開源版本之智財權與合規性，將老舊系統完全隔離、壓平並解耦成零依賴的 Stub 模擬系統 (`net.legacy.platform.core`)，以 MIT 開源授權條款 (MIT License) 釋出。

項目二：

跨平台檔案系統路徑防禦性批次檢索 **Cmdlet (Get-Files)** | 獨立開發者

專案網址：<https://github.com/johnny60904/get-files>

- 專案背景：為了消除辦公室日常大量檔案處理的低效與路徑錯誤風險，完全對齊 PowerShell 官方原生高階 Cmdlet (Advanced Cmdlet) 的語意和行為與生命週期鉤子 (Lifecycle Hooks)，開發出 Class-Based 的自動化核心。
- 防禦設計：具備高內聚的「跨平台檔案系統路徑格式防禦思維」，手工編寫出能同時辨識、防範、並清洗 Windows、Linux、MacOS 三大作業系統非法 File System Path 的結構防呆攔截器，阻止非法遍歷字元刺穿自動化管線 (Pipeline)。
- 效能優化：底層上鎖不可變性 Immutable (final/constant 紀律)，原生支援記憶體安全型緩衝串流 (Buffered Streaming)，並引入廣度優先 (BFS) 與深度優先 (DFS) 狀態機 (Finite-State Machine)，確保在進行多執行緒 (Multi-Thread)、數百萬級的深層目錄遞迴檢索時，系統具備「零記憶體配置溢出 (Anti-OOM)」的穩定防護力。

項目三：

基於抽象語法樹 (AST) 的自動化類別載入與靜態依賴關係分析工具 (Class Loader) | 獨立開發者

- 專案背景：為了解決複雜、大型自動化腳本專案中，手動維護 Cross-File 引用模組所造成的嚴重耦合與循環依賴痛點，直接深入直譯器底層所開發的核心分析引擎。
- 語法解析：直接對接 `System.Management.Automation.Language.Parser` 底層 API，在不執行任何未驗證代碼的「零安全風險」前提下，於語法樹 (AST) 碼點層級精準提取 Class 聲明、Interface 繼承、...等內部依賴特徵。
- 拓撲優化：手工實作拓撲排序演算法 (Topological Sorting)，將錯綜複雜的相互依賴物件編譯、路由成一條 100% 確定、無衝突的自動化腳本依賴加載順序鏈，為團隊大型自動化腳本奠定 CI/CD 靜態分析與自動化建置 (Automation Bootstrap) 的基石。

項目四：

多維度 **JSON Data Hardening & Serialization Security** 工具 (**Json Safe Converter**) | 獨立開發者

- 專案背景：為了解決跨網路未知 Request 負載在反序列化時，極易引發的記憶體耗盡 (OOM) 與飽和攻擊 (DoS / ReDoS) 等關鍵安全漏洞 (Security Vulnerabilities)。
- 多維防衛：採取「零信任邊界防禦 (Zero-Trust Boundary Defense)」，打造一系列具備 Strict / Balanced / Lenient 的多維度不變量限制 (Pillar Constraints)，包含最深巢狀限制 (Max Depth)、文字長度上限 (Max String Length)、與最大標記計數 (Max Token Count)、...等等。
- 併發絕緣：所有入站 Byte 負載在進行實體反序列化前，必須通過此一系列安全門檻的物理常數驗證。若超過安全門檻立即在最外層強力粉碎 (Throw Exception)，徹底隔絕了惡意 Hash 衝突或大量記憶體分配對主服務執行緒的資源飽和威脅。

教育背景與自主進修 / **EDUCATION & CONTINUOUS DEVELOPMENT**

獨立技術深潛與前沿 **AI** 應用實踐 (**Continuous Professional Development**)

2025 年 10 月 ~ 至今 (仍在進行中，每日維持 4 小時以上高強度自學)

- 架構與核心設計：全心投入資訊技術深水區。透過大量閱讀技術文件與實體複雜專案重構與實作，掌握了物件導向設計原則 (SOLID)、軟體工程模式 (CQRS / Strategy / Proxy)、領域驅動設計 (DDD) 的戰術設計與 Clean Architecture 戰略分層，養成極致乾淨的編碼紀律。
- **AI** 認知工程實戰：長期探索 Generative AI 的底層邏輯與提示工程邊界。發展出跨視窗狀態轉移 (State Transfer Prompting)、對話流控制、角色語意錨定 (Role Semantic Keywords) 與抽象層級控制等高密度脈絡控制技術，將 AI 轉化為 24 小時高 Velocity 的技術諮詢智囊與 workflow 推進器。

新北市立瑞芳高級工業職業學校 - 土木科 畢業

- 結構邏輯直覺養成：高職土木科的專業養成，奠定了我最初對於「結構應力、幾何空間、邊界約束」的物理邏輯直覺。如今我已將這套強大的「結構化與幾何解構思維」高度適應、移植到軟體架構分層、跨平台自動化工程與商業流程優化中，始終維持著依循科學法則做完善防禦與邊界控制的職人本性。

個人連結與 **GitHub / PORTFOLIO LINKS**

- **GitHub** 生產力原始碼與技術實驗庫：<https://github.com/johnny60904>
- 個人響應式雲端作品集 **RWD (Responsive Web Design) Portfolio**：<https://github.io>